

Lecture #1.

Pinocchio Introduction

Kyung Hee University
Team member: Eunae Kang, Chanjin Park
Prof. Sanghyun Kim



01

Introduction

Introduction | About the library

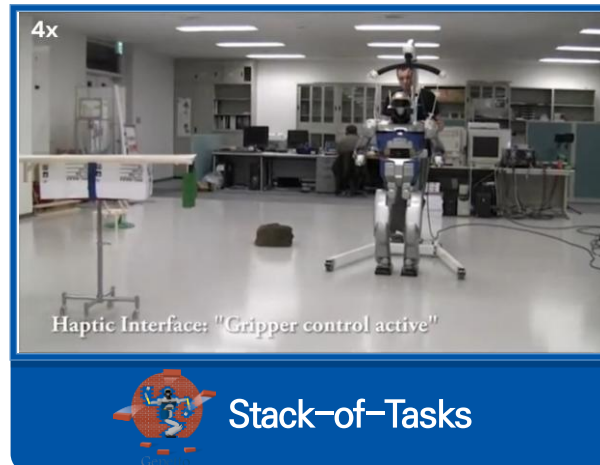
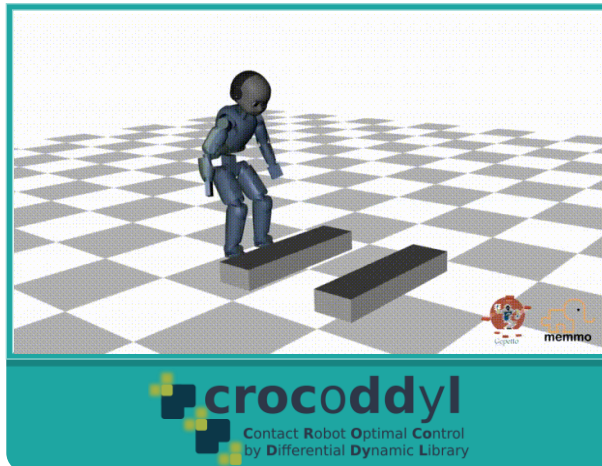
What is Pinocchio?

- An open-source software framework that implements rigid body dynamics algorithms and their analytical derivatives.

It includes

- Standard algorithms (e.g., forward and inverse dynamics)
- Additional features essential for the control
- The planning and the simulation of robots

Used research



Rigid Body Libraries

- The theory about Rigid Body Dynamics has been studied since 18th century
- There are two ways of implementation

Code-generation

- Model parameters are hard coded
- It allows simplification of math expression.
Therefore user can avoid unnecessary memory allocations or use of temporary variables.
- It suffers from a lack of flexibility; the code must be regenerated from scratch if any slight modification of the robot model occurs.

e.g. SD/Fast, ROBOTRAN, HuMANs, Symoro, METAPOD, RobCoGen

Dynamic frameworks

- Load a description file of the model at runtime
- It allows instance to modify the dynamical properties of the model or to populate the model with new features at runtime.
- It implies a loss in computational efficiency.

e.g. RBDL, OpenHRP, SymBody, RigidBodyDynamics.jl, Drake, Bullet, DART

- Pinocchio follows both paradigms.

Load any robot model at runtime (Dynamic frameworks)

Able to generate robot-specific source code (Code-generation)

What makes Pinocchio fast

Handling the sparsity

- In rigid body dynamics, each joint limits motion to certain directions.
Revolute joint: single scalar (rotation angle)
Spherical joint: 3*3 rotation matrix
- Pinocchio overloads spatial operators to handle computations efficiently.

Static polymorphism

- Uses the CRTP (Curiously Recurring Template Pattern); avoids dynamic redirection
- Uses the concept of *variant* to recover the flexibility

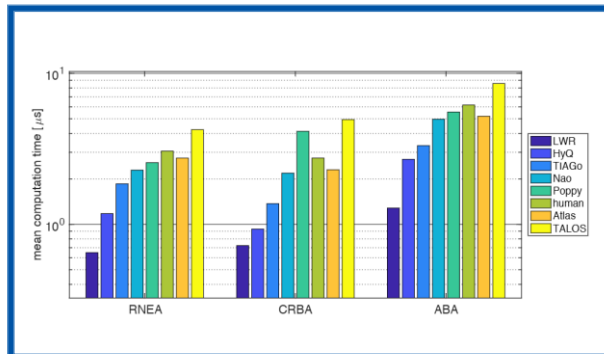
```
template<typename Derived>
struct JointModelBase<Derived> {
    void calc(q,v) {
        <call calc method of Derived class>
        static cast<Derived*>(this)→calc();
    }
};

struct JointModelRevolute: JointModelBase<JointModelRevolute> {
    void calc(q,v){
        <do specific computations>
    }
};
```

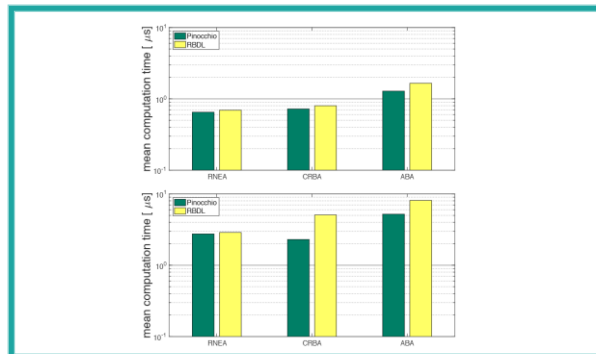
Introduction | About the library

Performance Comparison

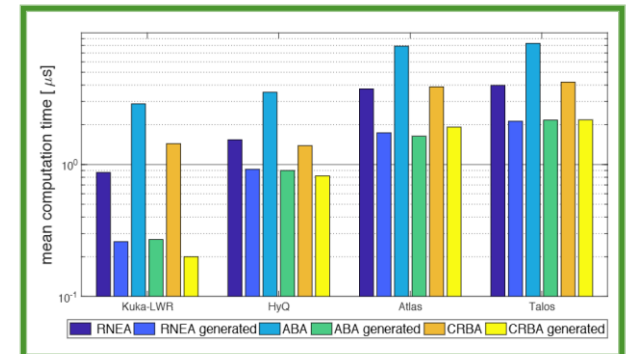
- Benchmark Setup: Intel i7 @ 2.4 GHz CPU
- Compared with RBDL about RNEA(inverse dynamics), ABA(forward dynamics), and CRBA(mass matrix) using 8 different robots.
- Pinocchio required about 1 μ s evaluating the dynamics on manipulator robots.
- It performs on par or faster than RBDL on dynamic loading.
- Computation time reduced 3–8x when a dedicated source code is generated.



Performance of Pinocchio
RNEA, ABA, CRBA



Comparison of performance of
Pinocchio against RBDL



Performances of code
Generated by Pinocchio

- Pinocchio bridges flexibility and speed.

Introduction | [How to install](#)

It is highly recommended to work in a virtual environment.

pip

- ▶ For Linux and Mac OS systems, Pinocchio is available on PyPI, and can be installed with its dependencies with:

```
python -m pip install pin
```

conda

- ▶ Pinocchio is available on conda forge, and can be installed with its dependencies with:

```
conda install pinocchio -c conda-forge
```

Linux

- Add robotpkg apt repository (if you have never added robotpkg as a softwares repository)

```
sudo apt install -qqy lsb-release curl
```

```
sudo mkdir -p /etc/apt/keyrings
curl http://robotpkg.openrobots.org/packages/debian/robotpkg.asc \
    | sudo tee /etc/apt/keyrings/robotpkg.asc
```

```
echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/robotpkg.asc]
http://robotpkg.openrobots.org/packages/debian/pub $(lsb_release -cs)
robotpkg" | sudo tee /etc/apt/sources.list.d/robotpkg.list
```

```
sudo apt update
```

Linux

- The installation of Pinocchio and its dependencies is made through the line:

```
sudo apt install -qqy robotpkg-py3*-pinocchio
```

Validation

- Check the installation of Pinocchio on terminal through the following command:

```
python <<'EOF'
import pinocchio

model = pinocchio.buildSampleModelManipulator()
data = model.createData()

q = pinocchio.neutral(model)
v = pinocchio.utils.zero(model.nv)
a = pinocchio.utils.zero(model.nv)

tau = pinocchio.rnea(model, data, q, v, a)
print("tau =", tau.T)
EOF
```

Spatial algebra

- Spatial algebra is a mathematical notation commonly employed in rigid body dynamics to represent and manipulate physical quantities such as velocities, accelerations and forces.
- Spatial velocity, Twist
- Spatial force, Wrench

$$\boldsymbol{\nu} = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \\ v_x \\ v_y \\ v_z \end{bmatrix} = \begin{bmatrix} \boldsymbol{\omega} \\ v \end{bmatrix}$$

$$\boldsymbol{F} = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \\ f_x \\ f_y \\ f_z \end{bmatrix} = \begin{bmatrix} \boldsymbol{\tau} \\ f \end{bmatrix}$$

Introduction | Main features

Model and data

- A fundamental paradigm of Pinocchio is the strict separation between model and data.

Code

```
robot = RobotWrapper.BuildFromURDF(urdf_model_path, mesh_dir)
model = robot.model

print("=== Pinocchio Model Info ===")
print("Number of joints:", model.njoints)
print("Number of frames:", model.nframes)
print("nq:", model.nq)
print("nv:", model.nv)

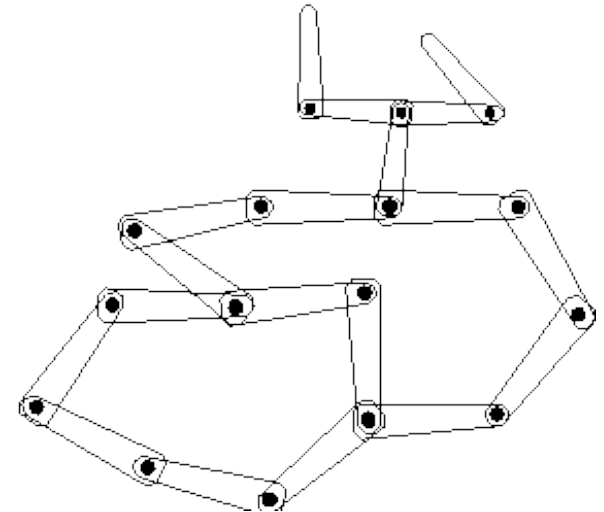
print("\n=== Joint List ===")
for idx, joint in enumerate(model.joints):
    print(f"[{idx}] {joint}, nq={joint.nq}, nv={joint.nv}")

print("\n=== Frame List ===")
for idx, frame in enumerate(model.frames):
    print(f"[{idx}] {frame.name}, type={frame.type}, parent={frame.parent}")

print("\n=== Inertia List ===")
for idx, inertia in enumerate(model.inertias):
    print(f"[{idx}] inertia mass={inertia.mass}, com={inertia.lever}")
```

- Model: fixed robot structure

- model = pinocchio.Model()
- model.name
- model.joints
- model.placements
- model.inertias
- model.frames



Introduction | Main features

Model and data

- A fundamental paradigm of Pinocchio is the strict separation between model and data.

Code

```
data = robot.data

print("=== Pinocchio Data Info ===")
print("\n=== Link Poses (oMi) ===")
for idx, Mi in enumerate(data.oMi):
    print(f"[{idx}] translation={Mi.translation.T}, rotation=\n{Mi.rotation}")

print("\n=== Mass Matrix (M) ===")
print(data.M)

print("\n=== Joint Velocities (v) ===")
for idx, vel in enumerate(data.v):
    print(f"Joint {idx}: {vel}")

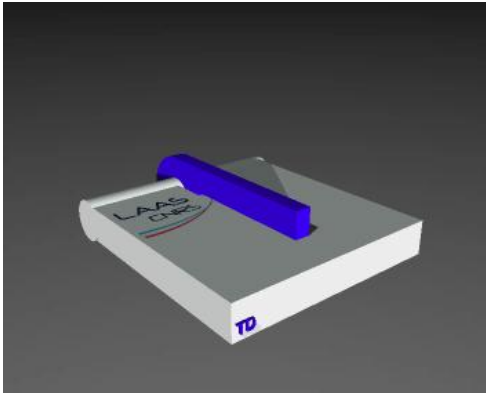
print("\n=== Joint Accelerations (a) ===")
for idx, acc in enumerate(data.a):
    print(f"Joint {idx}: {acc}")

print("\n=== Center of Mass (CoM) ===")
print("CoM position:", data.com[0].T)
print("CoM velocity:", data.vcom[0].T)
```

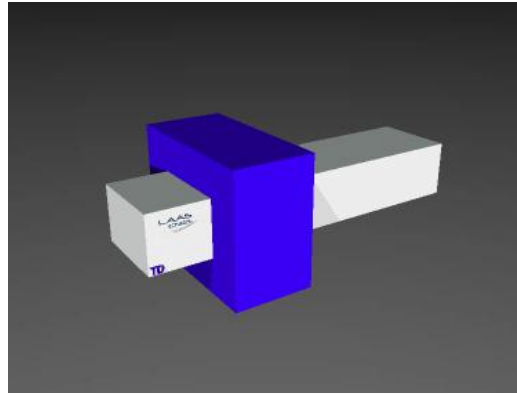
- Data: temporary values computed during algorithms
 - `data = pinocchio.Data(model) / data = model.createData()`
 - `data.joints`
 - `data.oMi`
 - `data.v`
 - `data.a`
 - `data.f`
 - `data.M`

Joints (Supported kinematic models)

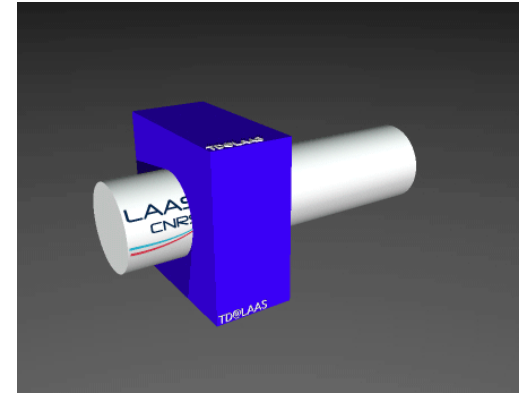
Revolute joint



Prismatic joint



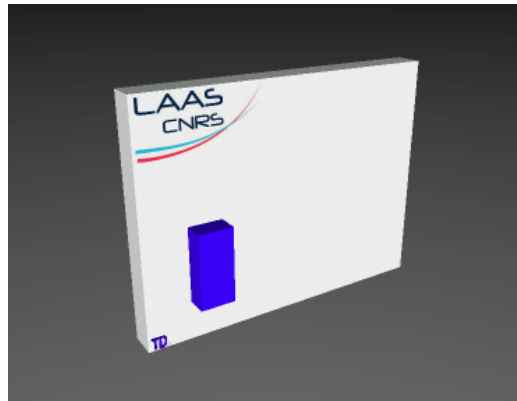
Cylindrical joint



Spherical joint



Planar joint



Geometric and collision models

- Pinocchio defines a geometric model, i.e. the volumes attached to the kinematic tree. This model can be used for displaying the robot and computing quantities associated to collisions.

Code

```
# Load model
model = pin.buildModelFromUrdf(urdf_model_path, pin.JointModelFreeFlyer())

# Load collision geometries
geom_model = pin.buildGeomFromUrdf(
    model, urdf_model_path, pin.GeometryType.COLLISION, mesh_dir
)

# Add collision pairs
geom_model.addAllCollisionPairs()
print("num collision pairs - initial:", len(geom_model.collisionPairs))

# Remove collision pairs listed in the SRDF file
srdf_filename = "romeo.srdf"
srdf_model_path = model_path / "romeo_description/srdf" / srdf_filename

pin.removeCollisionPairs(model, geom_model, srdf_model_path)
print(
    "num collision pairs - after removing useless collision pairs:",
    len(geom_model.collisionPairs),
)

# Load reference configuration
pin.loadReferenceConfigurations(model, srdf_model_path)

# Retrieve the half sitting position from the SRDF file
q = model.referenceConfigurations["half_sitting"]
```

Geometric model

- `geom_model.addAllCollisionPairs()`
- `geom_model.collisionPairs`
- `pinocchio.removeCollisionPairs(model, geom_model, srdf_model_path)`

Geometric and collision models

- Pinocchio defines a geometric model, i.e. the volumes attached to the kinematic tree. This model can be used for displaying the robot and computing quantities associated to collisions.

Code

```
# Create data structures
data = model.createData()
geom_data = pin.GeometryData(geom_model)

# Compute all the collisions
pin.computeCollisions(model, data, geom_model, geom_data, q, False)

# Print the status of collision for all collision pairs
for k in range(len(geom_model.collisionPairs)):
    cr = geom_data.collisionResults[k]
    cp = geom_model.collisionPairs[k]
    print(
        "collision pair:",
        cp.first,
        ",",
        cp.second,
        "- collision:",
        "Yes" if cr.isCollision() else "No",
    )

# Compute for a single pair of collision
pin.updateGeometryPlacements(model, data, geom_model, geom_data, q)
pin.computeCollision(geom_model, geom_data, 0)
```

Geometric data

- `geom_data = pinocchio.GeometryData(geom_model)`
- `pinocchio.computeCollisions(model, data, geom_model, geom_data, q)`
- `pinocchio.collisionResults`
- `pinocchio.updateGeometryPlacements(model, data, geom_model, geom_data, q)`
- `pinocchio.computeCollision(geom_model, geom_data, 0)`

Introduction | Main features

Dealing with Lie-group geometry

Pinocchio relies heavily on Lie groups and Lie algebras to handle motions and more specifically rotations.

Special Orthogonal Group

$$SO(n) = \{R \in \mathbf{R}^{n \times n} : RR^T = I, \det(R) = 1\}$$

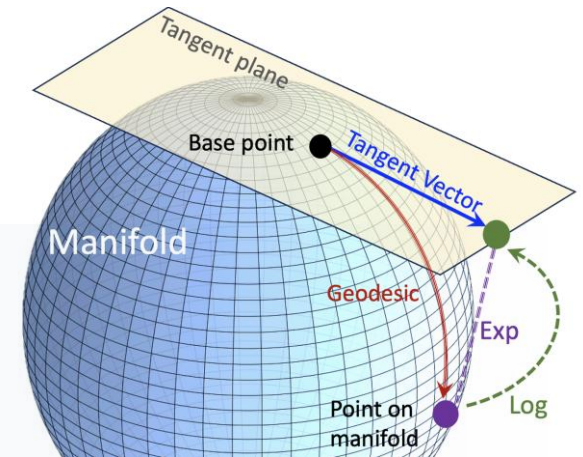
Special Euclidean Group

$$SE(3) = \{(R, p) : R \in SO(3), p \in \mathbf{R}^3\} = SO(3) \times \mathbf{R}^3$$

```
pose_s_3d = pin.SE3.Random()
print(pose_s_3d)
print(pose_s_3d.rotation, pose_s_3d.translation)
```

```
→ SE3(array([[ 0.0452059 , -0.99506743, -0.08830191,  0.02680182], [-0.03981301,
 0.08652748, -0.99545362,  0.90445945], [ 0.99818403,  0.04851594, -0.03570508,
 0.83239014], [ 0. ,  0. ,  0. ,  1. ]]))
```

```
(array([0.02680182, 0.90445945, 0.83239014]),
 array([[ 0.0452059 , -0.99506743, -0.08830191],
 [-0.03981301,  0.08652748, -0.99545362],
 [ 0.99818403,  0.04851594, -0.03570508]]))
```



Dealing with Lie-group geometry

Rotations	Rigid-Body Motions
$R \in SO(3) : 3 \times 3$ matrices $R^T R = I, \det R = 1$	$T \in SE(3) : 4 \times 4$ matrices $T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$, where $R \in SO(3), p \in \mathbb{R}^3$
$R^{-1} = R^T$	$T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}$
change of coordinate frame: $R_{ab}R_{bc} = R_{ac}, R_{ab}p_b = p_a$	change of coordinate frame: $T_{ab}T_{bc} = T_{ac}, T_{ab}p_b = p_a$
rotating a frame $\{b\}$: $R = \text{Rot}(\hat{\omega}, \theta)$ $R_{sb'} = RR_{sb}$: rotate θ about $\hat{\omega}_s = \hat{\omega}$ $R_{sb''} = R_{sb}R$: rotate θ about $\hat{\omega}_b = \hat{\omega}$	displacing a frame $\{b\}$: $T = \begin{bmatrix} \text{Rot}(\hat{\omega}, \theta) & p \\ 0 & 1 \end{bmatrix}$ $T_{sb'} = TT_{sb}$: rotate θ about $\hat{\omega}_s = \hat{\omega}$ (moves $\{b\}$ origin), translate p in $\{s\}$ $T_{sb''} = T_{sb}T$: translate p in $\{b\}$, rotate θ about $\hat{\omega}$ in new body frame
unit rotation axis is $\hat{\omega} \in \mathbb{R}^3$, where $\ \hat{\omega}\ = 1$	“unit” screw axis is $\mathcal{S} = \begin{bmatrix} \omega \\ v \end{bmatrix} \in \mathbb{R}^6$, where either (i) $\ \omega\ = 1$ or (ii) $\omega = 0$ and $\ v\ = 1$
	for a screw axis $\{q, \hat{s}, h\}$ with finite h , $\mathcal{S} = \begin{bmatrix} \omega \\ v \end{bmatrix} = \begin{bmatrix} \hat{s} \\ -\hat{s} \times q + h\hat{s} \end{bmatrix}$
angular velocity is $\omega = \dot{\omega}\theta$	twist is $\mathcal{V} = \mathcal{S}\dot{\theta}$

Rotations (cont.)	Rigid-Body Motions (cont.)
for any 3-vector, e.g., $\omega \in \mathbb{R}^3$, $[\omega] = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix} \in so(3)$ identities, $\omega, x \in \mathbb{R}^3, R \in SO(3)$: $[\omega] = -[\omega]^T, [\omega]x = -[x]\omega$, $[\omega][x] = ([x][\omega])^T, R[\omega]R^T = [R\omega]$	for $\mathcal{V} = \begin{bmatrix} \omega \\ v \end{bmatrix} \in \mathbb{R}^6$, $[\mathcal{V}] = \begin{bmatrix} [\omega] & v \\ 0 & 0 \end{bmatrix} \in se(3)$ (the pair (ω, v) can be a twist $\mathcal{V}$ or a “unit” screw axis $\mathcal{S}$, depending on the context)
$\dot{R}R^{-1} = [\omega_s], R^{-1}\dot{R} = [\omega_b]$	$\dot{T}T^{-1} = [\mathcal{V}_s], T^{-1}\dot{T} = [\mathcal{V}_b]$
	$[\text{Ad}_T] = \begin{bmatrix} R & 0 \\ [p]R & R \end{bmatrix} \in \mathbb{R}^{6 \times 6}$ identities: $[\text{Ad}_T]^{-1} = [\text{Ad}_{T^{-1}}]$, $[\text{Ad}_{T_1}][\text{Ad}_{T_2}] = [\text{Ad}_{T_1 T_2}]$
change of coordinate frame: $\hat{\omega}_a = R_{ab}\hat{\omega}_b, \omega_a = R_{ab}\omega_b$	change of coordinate frame: $\mathcal{S}_a = [\text{Ad}_{T_{ab}}]\mathcal{S}_b, \mathcal{V}_a = [\text{Ad}_{T_{ab}}]\mathcal{V}_b$
exp coords for $R \in SO(3)$: $\hat{\omega}\theta \in \mathbb{R}^3$	exp coords for $T \in SE(3)$: $\mathcal{S}\theta \in \mathbb{R}^6$
exp : $[\hat{\omega}]\theta \in so(3) \rightarrow R \in SO(3)$ $R = \text{Rot}(\hat{\omega}, \theta) = e^{[\hat{\omega}]\theta} =$ $I + \sin \theta [\hat{\omega}] + (1 - \cos \theta) [\hat{\omega}]^2$	exp : $[\mathcal{S}]\theta \in se(3) \rightarrow T \in SE(3)$ $T = e^{[\mathcal{S}]\theta} = \begin{bmatrix} e^{[\omega]\theta} & * \\ 0 & 1 \end{bmatrix}$ where $*$ = $(I\theta + (1 - \cos \theta)[\omega] + (\theta - \sin \theta)[\omega]^2)v$
log : $R \in SO(3) \rightarrow [\hat{\omega}]\theta \in so(3)$ algorithm in Section 3.2.3.3	log : $T \in SE(3) \rightarrow [\mathcal{S}]\theta \in se(3)$ algorithm in Section 3.3.3.2
moment change of coord frame: $m_a = R_{ab}m_b$	wrench change of coord frame: $\mathcal{F}_a = (m_a, f_a) = [\text{Ad}_{T_{ba}}]^T \mathcal{F}_b$

Main algorithms

- ◉ Kinematic algorithms
 - Forward kinematics
 - Kinematic jacobian
- ◉ Dynamics algorithms
 - Inverse dynamics
 - Joint-space inertia matrix
 - Forward dynamics

Analytical derivatives

- ◉ These derivatives are for instance of primary importance in the context of whole-body trajectory optimization or more largely, for numerical optimal control.

Automatic differentiation and source code generation

- ◉ Pinocchio supports AD through templated scalar types and libraries like ADOL-C, CasADi, and CppAD, but AD is slower than its built-in analytical derivatives.
- ◉ Pinocchio can generate optimized C (and other) code at compile time or runtime, producing fast, robot-specific code that can run independently.

Thank You